Designed by Remus Teodorescu 2026

LEARNING OBJECTIVES

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **frame** — a driving task as classification or as regression, and say which fits
- **collect** — a dataset that will still work when the lighting changes
- **reuse** — a pretrained backbone, and remember the normalisation
- **compute** — a latency budget, and say what it costs in centimetres
- **convert** — a trained model for the device, and measure the gain
- **name** — what a reactive policy cannot do, and what L11.2 adds

WHAT TODAY ASSUMES

from L5.1	convolutional networks
from L6.2	transfer learning, and quantisation
from L6.2	imitation against reinforcement

NO PDE TODAY

This is the one lecture in Part 2 with no differential equation in it. The physics returns in L11.2, and it is not a PDE either.

AND ONE NEW CURRENCY

Latency. A model that is more accurate and slower is usually worse.

NVIDIA JetRacer; Waveshare docs

ORIENTATION

The One Lecture Without a PDE

SAY THIS PLAINLY

There is no governing equation here. The network in L11.1 does pattern matching on pixels, with no model of the world, no residual, and nothing to verify against. Presenting it as physics-informed would be dishonest — the physics returns in L11.2.

	L7–L10	L11.1
Constraint	a PDE residual	labelled images
Verification	manufactured solution	held-out data, then the floor
What limits you	conditioning, sampling	latency and data
Failure	loss stalls	the car hits a wall
Where physics returns	—	L11.2: dynamics and energy

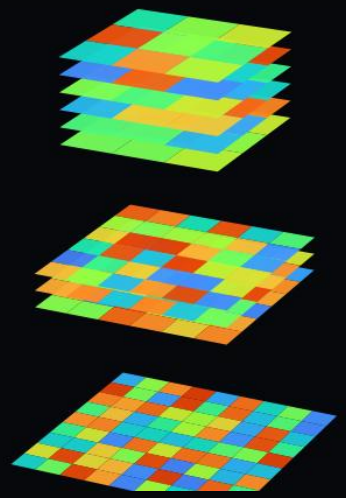
You already know CNNs from the earlier part of the course. What is new here is that the model runs on a moving vehicle, and the world does not wait for it.

BACK TO PART 1

Why a Convolutional Network, and Not Any Other

WHAT L5.1 SAID, ARRIVING ON HARDWARE

- **local receptive fields** — a wall is a local pattern, wherever it sits in the frame
- **weight sharing** — one edge detector serves the whole image, so the count stays small
- **translation equivariance** — an obstacle left of centre and right of centre look alike



WHAT THE JETRACER USES

ResNet-18	18 layers, residual connections
from L5.1	convolution, pooling, stride
from L4.2	why depth beats width
new here	the last layer, and nothing else

THE RESIDUAL CONNECTION

L4.2 said deep networks are hard to train because gradients vanish. A skip connection is the answer, and it is what the 'Res' means.

AND WHY NOT A PLAIN MLP

A fully connected layer on a 224 x 224 image needs millions of weights for the first layer alone.

The JetRacer Pro, Component by Component

	WHAT	WHY IT MATTERS
Compute	Jetson Nano 4GB	Cortex-A57 CPU, 128-core Maxwell GPU, 472 GFLOPS
Camera	8 MP, 160° FOV	very wide angle — heavy barrel distortion
Drive	4WD, front and rear differentials	Ackermann steering via a front servo
Power	4 × 18650, 8.4 V (2S2P)	buck-boost regulator supplies a stable 5 V
Telemetry	OLED + INA219	displays IP; monitors battery voltage and current
Networking	AC8265 dual-band	you program it from a browser, over WiFi

Note the last-but-one row. The INA219 measures the car's own power draw — which is what makes the energy MiniProject an experiment rather than a simulation.

Getting a Car Moving, and Two Traps

1 Flash — Write the Waveshare JetRacer image to a ≥ 64 GB SD card with Etcher. The image is pre-built on JetPack 4.5 — do not build your own.

2 Boot — Insert the card, power on, wait. The OLED shows the car's IP address once it has joined the WiFi.

3 Connect — Open http://<ip>:8888 in a browser. JupyterLab runs on the car; you write code in a browser and it executes on the Nano.

4 Drive — Open /jetracer/notebooks/ and run basic_motion.ipynb. If the wheels turn, the stack is working. Do this before anything else.

JetRacer Pro does not support the Orin series — different power architecture, incompatible project. These six cars are what they are.

TWO TRAPS THAT WILL COST YOU A SESSION

1. Waveshare code, not NVIDIA's

The motor drive code differs between the two projects. If you follow an NVIDIA JetRacer tutorial and overwrite the jetracer folder, the car will not drive. Remove it and reinstall the Waveshare version.

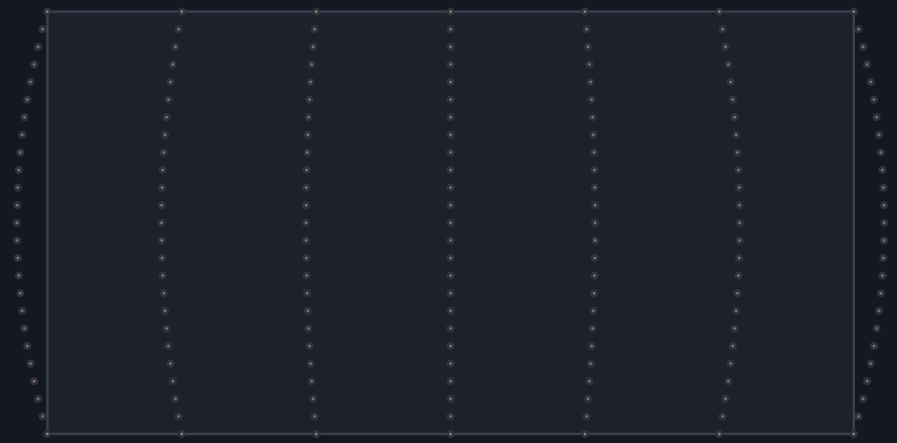
2. 5 W power mode

The Nano must be placed in 5 W mode so it does not draw more current than the battery pack can supply. Skip this and the car resets under load — usually mid-demonstration.

What the Network Actually Sees

- **a 160° field of view is very wide** — it buys peripheral awareness
- the network does not care that the image is distorted
- provided the distortion is the same in training and in deployment
- so mount the camera once and never move it
- **and downscale** — 224 × 224 is far more than the task needs

WIDE ANGLE: COVERAGE AGAINST DISTORTION



straight lines bow outward at the edges

Mount once. A camera moved by five degrees invalidates every training image you have, and nothing will warn you.

Waveshare hardware spec; practical imaging

PERCEPTION

Two Tasks, Two Output Layers

BOTH ARE THE SAME THING: AN IMAGE GOES IN, AN ACTION COMES OUT

$$\mathbf{a}_t = \pi_{\hat{w}}(\mathbf{I}_t)$$

CLASSIFY

Collision avoidance

Two classes: free or blocked.
Output a probability; act on a threshold.
Labelling is fast — press one of two buttons.
Where Ex_11.1 starts.

REGRESS

Road following

Output a target point in the image to steer towards.
Labelling is slower — click the point in each frame.
Gives smooth control rather than stop/go.

- **start with classification** — two buttons and a few hundred images

Obstacle Avoidance as Binary Classification

TWO CLASSES

$$p(\text{blocked} \mid \mathbf{I}) = softmax\big(f_{\hat{w}}(\mathbf{I})\big)$$

CROSS-ENTROPY LOSS

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k} y_{ik}\log p_{ik}$$

- the label is not 'is there an obstacle' but 'would driving forward be safe'
- **collect both classes in the same conditions** — same floor, same height
- otherwise the network learns the lighting, not the obstacle
- and threshold deliberately: the two errors are not equal

REPORT BOTH

$$acc = \frac{TP + TN}{N}, \qquad recall_{blocked} = \frac{TP}{TP + FN}$$

THE TWO ERRORS ARE NOT EQUAL

	predicted free	predicted blocked
truly free	correct	stops needlessly
truly blocked	**COLLISION**	correct

Tune the threshold so the bottom-left cell is as close to empty as you can make it, and accept the cost in the top-right.

Road Following as Regression

PREDICT WHERE TO STEER TOWARDS

$$(\hat{x},\, \hat{y}) = f_{\hat{w}}(\mathbf{I})$$

- instead of a class, the network outputs image coordinates
- the output is continuous, so the control is smooth
- a classifier can only start and stop; a regressor can steer
- the steering command is a controller on the predicted offset
- and the same backbone serves both tasks

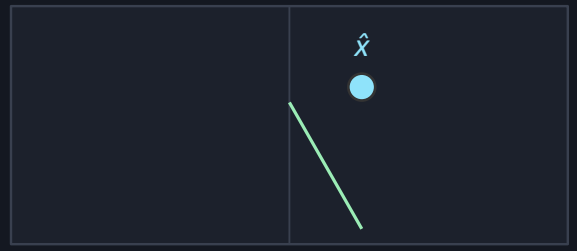
SQUARED ERROR ON THE TARGET POINT

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N} \| \, (\hat{x}_i, \hat{y}_i) - (x_i, y_i) \, \|^2$$

STEERING FROM THE OFFSET

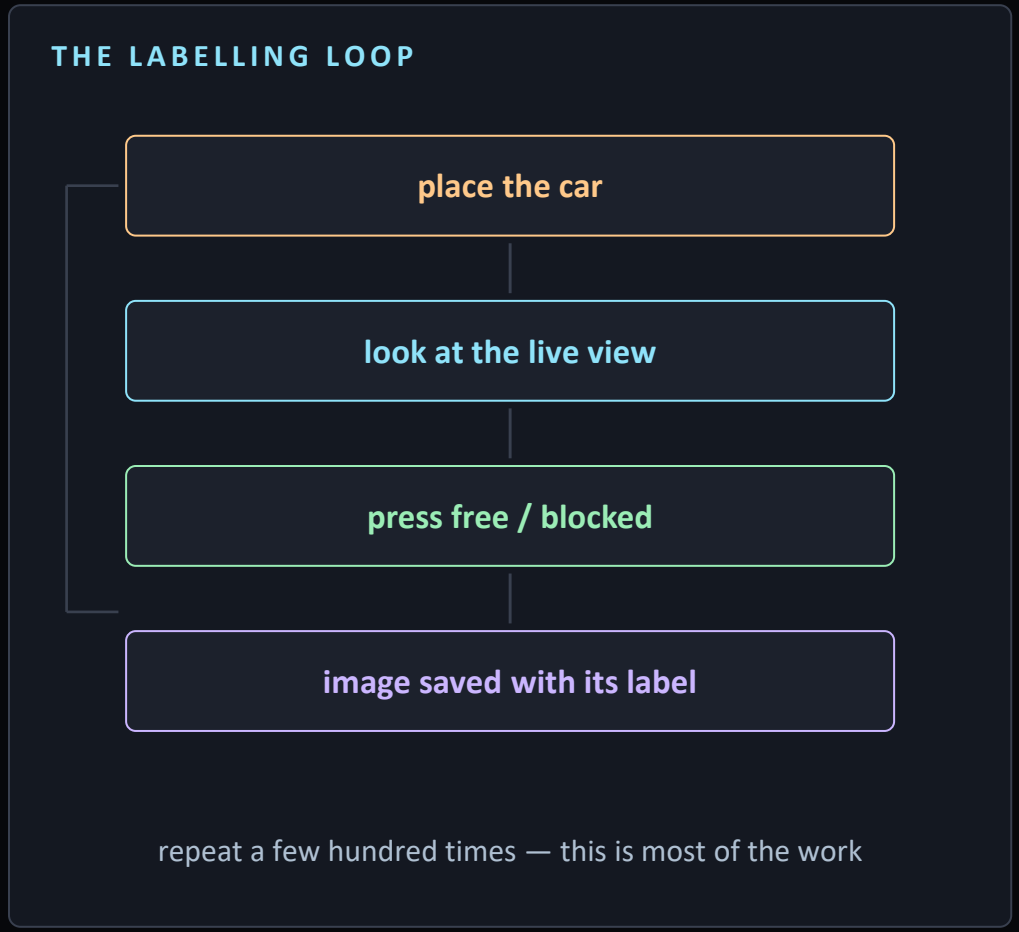
$$\delta = K_p \, \hat{x} + K_d \, \frac{d\hat{x}}{dt}$$

THE PREDICTED TARGET



Collecting Data Is the Exercise

- the reference notebooks put a live feed and labelling buttons in the browser
- collect where the car will actually drive
- balance the classes and vary everything else
- a few hundred images per class is enough to start
- **and deliberately include recovery states** — half off the line, facing a wall



Transfer Learning: Reuse the Backbone

A NEW HEAD ON A PRETRAINED BODY

$$f_{\hat{w}} = h_{new} \circ g_{pretrained}$$

USE THE STATISTICS THE BACKBONE WAS TRAINED WITH

$$\mathbf{I}^* = \frac{\mathbf{I}/255 - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \qquad \boldsymbol{\mu}, \boldsymbol{\sigma}\ from\ ImageNet$$

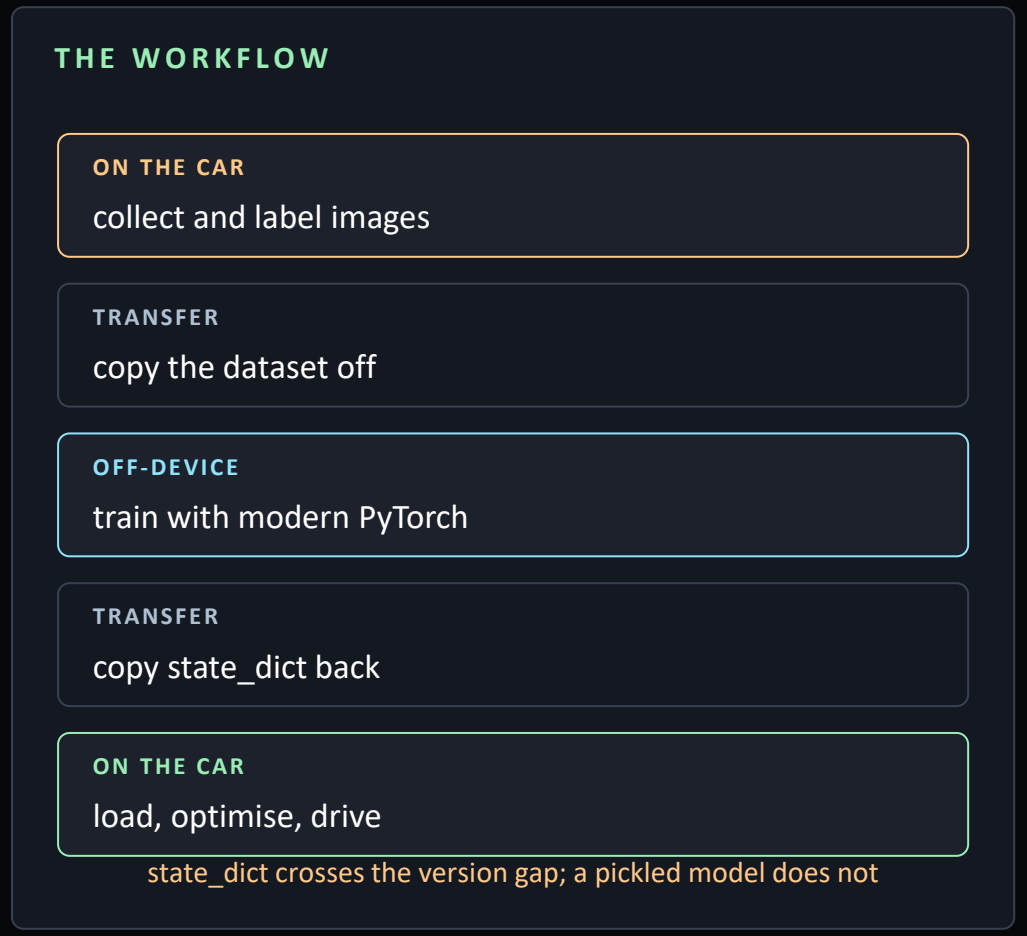
- a ResNet-18 pretrained on ImageNet already detects edges, textures and shapes
- replace only the final fully connected layer
- **normalise with the ImageNet statistics** — forgetting this is the commonest error
- with a few hundred images, fine-tune the whole network at a low rate

You met CNNs earlier in the course. Nothing here is new architecture — what is new is that the result has to run on a battery-powered car at speed.

Standard practice; JetRacer reference notebooks

Train Off-Device, Deploy On-Device

- **the Nano runs an image built on JetPack 4.5** — Python 3.6, and a PyTorch of that era
- so train elsewhere, with modern tooling, and deploy the weights
- transfer the state dictionary, not the pickled model
- a pickled object carries class paths that will not resolve
- and keep the architecture definition identical on both sides



Latency, Not Accuracy

CONTROL LOOP RATE

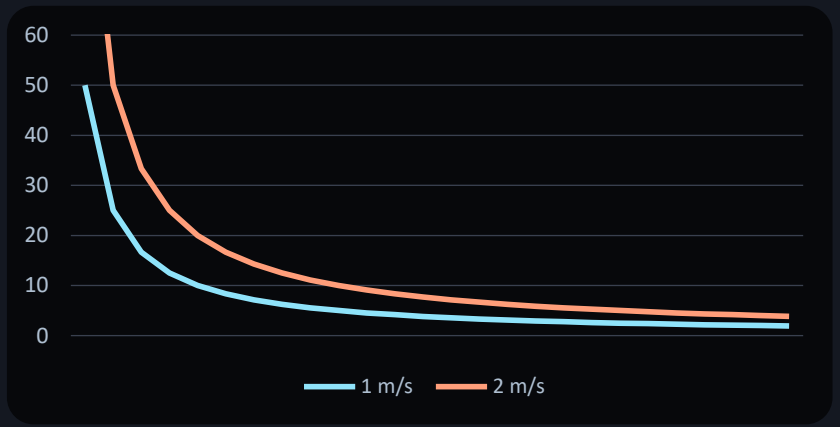
$$f_{loop} = \frac{1}{t_{capture} + t_{infer} + t_{actuate}}$$

DISTANCE TRAVELLED BETWEEN DECISIONS

$$d_{blind} = \frac{v}{f_{loop}}$$

- between one inference and the next, the car is driving blind
- at 2 m/s and 10 frames per second that is 20 cm
- this lesson has no analogue anywhere earlier in the course
- in L7 to L10 a slow model was merely inconvenient
- so measure latency on the car, not on your laptop

BLIND DISTANCE



cm travelled between decisions, against frames per second

THE REAL CONSTRAINT

TensorRT: Where the Frame Rate Comes From

- torch2trt fuses layers, picks kernels, and can drop to half precision
- half precision is the large win on a Maxwell GPU
- accuracy loss for free-or-blocked is usually negligible
- convert after training, on the car, and measure before and after
- the conversion is device-specific and does not transfer

WHAT TO MEASURE AND REPORT

accuracy held-out set	%
recall (blocked) held-out set	%
inference time on the car, 5 W mode	ms
loop rate capture + infer + actuate	Hz
blind distance speed / loop rate	cm

every report in Ex_11.1 quotes all five

torch2trt; JetRacer stack

THE REAL CONSTRAINT

The Trade-Off, Stated Plainly

A BIGGER MODEL SEES BETTER AND REACTS LATER

$$accuracy \uparrow \;\; but \;\; f_{loop} \downarrow \Rightarrow \;\; d_{blind} \uparrow$$

RESNET-18

The reference choice. Accurate enough, and fast enough on a Nano once converted.

SOMETHING SMALLER

MobileNet or a custom shallow network. Lower accuracy, much higher frame rate. Often the better car.

SOMETHING LARGER

ResNet-50 and beyond. Better on the held-out set, worse on the floor. Try it once, so you have measured it.

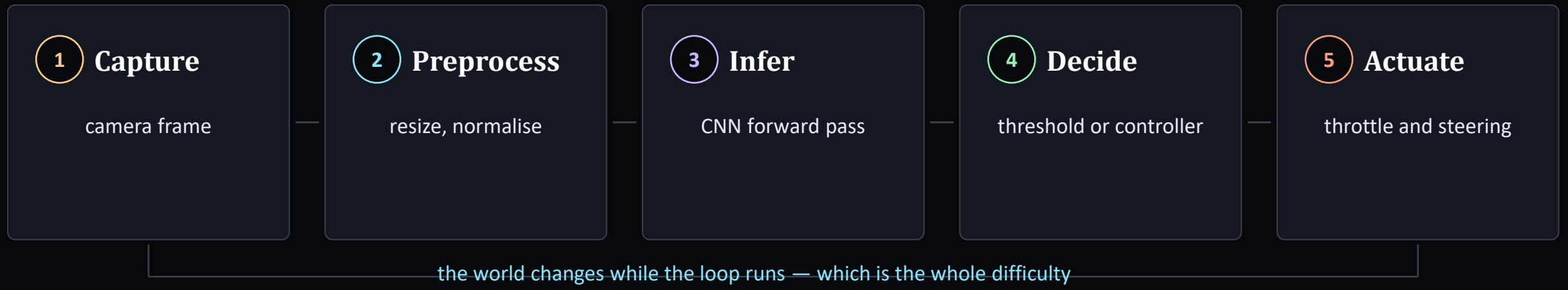
LOWER RESOLUTION

Halving the input side quarters the work. Frequently the cheapest speed-up available, and the first thing to try.

The car is the benchmark. A model that scores worse on the held-out set and drives the course faster is the better model, and saying so is the point of the exercise.

CONTROL

Closing the Loop



- every stage costs time, and only one of them is the network
- students routinely optimise the model and leave the camera pipeline untouched
- **add hysteresis to the decision** — a raw threshold makes the car stutter

LIMITS

How These Models Fail

It learned the room	Free and blocked images collected in different places, so the network keys on the carpet rather than the obstacle.	Symptom: perfect held-out accuracy, useless in a new room. Fix: collect both classes in the same place.
Lighting changed	Trained in the afternoon, demonstrated in the evening with the blinds down.	Fix: collect across lighting conditions, or augment brightness and contrast.
Never saw recovery	Training data contains only good driving, so the model has never seen the car pointing at a wall.	Fix: deliberately collect from bad positions. This is the single highest-value fix.
Too slow to matter	Accurate model, loop running at 4 Hz, car travelling half a metre between decisions.	Fix: TensorRT, smaller input, smaller backbone. Measure the loop, not the model.
Camera moved	The mount shifted — in transit, in a collision, or because someone adjusted it.	Symptom: sudden unexplained degradation. Check the mount before retraining anything.

The first and third are the ones that catch everyone. Both are data-collection errors, and neither is fixable by training longer.

PRACTICE

Lab Safety and Working Habits

A hand on the stop	Someone other than the driver watches the car with the kill switch ready. Six cars in one room is six chances for a runaway.
Speed limit in software	Cap the throttle in code during development. Raise it only once the loop is measured and the car behaves at the lower speed.
Battery discipline	Lithium cells: charge with the supplied charger only, never leave charging unattended, never run a pack flat. The kit warns about this and it is not boilerplate.
Label your car	Six identical cars, six SD cards, six datasets. Number them and keep each team on one car — mixed-up SD cards waste a session.
Version the weights	Save the model with the date, the dataset it came from, and the measured accuracy and frame rate. You will otherwise not know which file drove the good lap.

The battery rule is the one that matters most. Everything else costs a session; that one can cost a laboratory.

EXERCISE

Ex_11.1 — Get a Car Avoiding Obstacles

TASKS

1. Flash, boot and connect. Run basic_motion.ipynb and confirm the wheels turn, in 5 W mode, on Waveshare code.
2. Collect a balanced free/blocked dataset on the floor you will drive, including recovery viewpoints.
3. Train a ResNet-18 off-device, transfer the state dict back, and verify it loads.
4. Measure inference time and loop rate before and after TensorRT conversion.
5. Drive a fixed course. Report accuracy, recall on blocked, loop rate and blind distance.

THE CAR, AND WHAT IT CANNOT DO

compute	Jetson Nano 4 GB, 128-core Maxwell
camera	IMX219-160, 8 MP, 160 degree FOV
drive	RC380 brushed, 4WD, no encoder
pack	4 x 18650, 8.4 V, 2S2P

WHAT THE STOCK CAR CANNOT DO

One speed always. No idea of its own latency. No failsafe on a stale frame. And nothing stops it commanding more grip than the tyres have.

Works on an open floor with cardboard obstacles. A taped line turns the same pipeline into road following — change the head and the loss, nothing else.

LOOKING AHEAD

Where the Physics Comes Back

- **everything in L11.1 is reactive** — a frame in, a command out
- no model of the car, no notion of energy, no constraint on what is achievable
- L11.2 adds all three, and none of them is a PDE

WHAT L11.2 ADDS

kinematic bicycle model	**how it moves**
friction, steering limits	**what is impossible**
motor and battery model	**what it costs**
INA219 measurement	**what it actually cost**
energy-optimal control	**the MiniProject**

the fourth row is why this can be measured, not just modelled

SUMMARY

Key Takeaways

1 No physics here, and say so	L11.1 is pattern matching on pixels. There is no residual and nothing to verify against but held-out data and the floor. The physics returns in L11.2.
2 Latency is the constraint	Between decisions the car drives blind. A model three percent better and half as fast is usually a worse car — quote accuracy and frame rate together, always.
3 Data collection is the work	Same place, both classes, and deliberately including the bad positions the car reaches only when already going wrong. Most failures are collection errors.
4 Reuse the backbone	A pretrained ResNet-18 with a replaced head and ImageNet normalisation. Training from scratch on a few hundred images will not work.
5 The car is the benchmark	Held-out accuracy is a proxy. The measurement that counts is whether it completes the course, and how fast.

Next — L11.2: dynamics, energy, and the MiniProject where the car has to drive efficiently as well as safely.

REFERENCES

Where to Look

Waveshare JetRacer Pro AI Kit wiki	The primary source: image download, assembly, startup, and the warning that Waveshare motor code differs from NVIDIA's. Start here and follow it exactly.
The on-car notebooks	/jetracer/notebooks/ on the car itself. basic_motion.ipynb first, then the data-collection and training notebooks. These are the working reference, not a tutorial to be adapted.
NVIDIA JetBot and JetRacer projects	Useful for understanding the approach — collision avoidance as classification, road following as regression. Read for method; do not copy the motor code.
torch2trt	The PyTorch-to-TensorRT translator that supplies the frame rate. Conversion is device-specific: build the engine on the car it will run on.
Your earlier CNN lecture	Architectures, transfer learning and augmentation are assumed knowledge here. This lecture adds only the constraint of running on a moving vehicle.

No academic references for this lecture: it is an engineering session, and the vendor documentation is the authority.

OVER TO YOU

Questions

TEN TO TEST WHETHER TODAY LANDED

- what does the label actually mean in obstacle avoidance?
- why is regression smoother to control than classification?
- why must training and deployment share the same camera mount?
- what does the ImageNet normalisation do, and what breaks without it?
- at 2 m/s and 4 fps, how far does the car travel blind?
- why convert on the car rather than on your laptop?

AND FOUR MORE

seven	why collect recovery states deliberately?
eight	why transfer the state dictionary, not the model?
nine	which stage of the loop do students forget to time?
ten	what can a reactive policy not do?

THE ONE WITH NO SETTLED ANSWER

Your model drifts off the line. More data, or a different formulation?

BEFORE L11.2

Ex_11.1. Bring your measured frame rate and blind distance.

COMPETITION

The Navigation Trophy — Six Cars, One Course

EVERY CAR IS SCORED AGAINST ITSELF

Before the session each car runs the stock road-following policy at fixed throttle, and its median lap time becomes that car's baseline T_0. Your score is a ratio against your own car, so a tired pack or a worn tyre costs you nothing. Cars are drawn at random.

THE NAVIGATION INDEX

$$N = T_0 / (T + 2c)$$

T is the median of three consecutive clean laps; c is the number of wall contacts across those laps, each charged at two seconds. Higher N is better, and N > 1 means you beat the car you were given.

Three laps, not one. A single fast lap is a sample of size one, and this course does not reward luck.

THE LATENCY DECLARATION

$$d = v / f$$

Every team reports its measured loop rate f and the resulting blind distance d at competition speed. This is the quantity from slide 12, and it is not optional.

A team whose blind distance exceeds half the narrowest clearance on the course is capped at N = 1.00 however fast it went. Driving beyond what you can see is not a result, it is a near miss.

GATES — FAIL ANY ONE AND THE RUN DOES NOT SCORE

- **Completes** three consecutive laps, no manual intervention
- **Reports f** loop rate measured on the car, not assumed
- **Failsafe** stale frame demonstrably stops the car
- **Held out** model validated on data it was not trained on

AND IT CARRIES FORWARD

Your T_0, your loop rate and your held-out error are recorded and reused in Ex_11.2. The team that wins here starts the energy competition with a policy worth optimising.